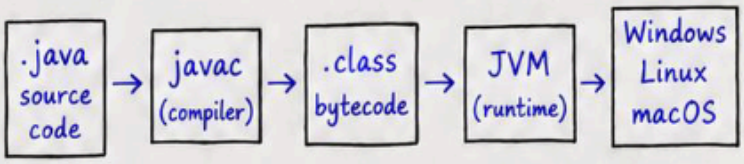
Java Important Cheat Sheet

Java Fundamentals

1. What is Java?

Object-oriented, strongly typed and platform-independent programming language.

2. Write Once, Run Anywhere



3. JDK vs JRE vs JVM

Component	Purpose
JDK	Develop + compile + run (includes JRE)
JRE	Libraries + JVM (to run Java programs)
JVM	Executes bytecode (platform-specific)

4. Basic Program

```
public class Main {  ← class name
    public static void main(String[] args) {
                                ↑
                           entry point
        System.out.println("Hello, Java!");
                                ↑
    }                    output statement
}
```

5. Output

```
System.out.print("Hi");   → Hi
System.out.println("Java"); → Java
System.out.printf("Age: %d", 25);
                  → Age: 25
```

6. Comments

```
// single-line comment
   (ignored by compiler)

/* multi-line comment
   can span multiple lines
 */

/** documentation comment
    used for generating API docs
 */
```

7. Important Notes

- File name must match public class name (e.g., Main.java for class Main).
- Java is case-sensitive (e.g., Hello and hello are different).
- Statements usually end with a semicolon (;).

Java Important Cheat Sheet

Variables, Data Types & Casting

1. Primitive Types

Type	Description
byte	8-bit integer
short	16-bit integer
int	32-bit integer
long	64-bit integer
float	32-bit decimal
double	64-bit decimal
char	16-bit Unicode
boolean	true / false

2. Declaration

```
int age = 25;
double price = 99.5;
char grade = 'A';
boolean active = true;
String name = "Aman";
```

3. Constants & var

```
final double PI = 3.14159;
var message = "Hello";
  // local variable inference
```

4. Type Casting

Widening (implicit casting)

```
int x = 10;
double y = x;   // int → double
```

Narrowing (explicit casting)

```
double a = 9.8;
int b = (int) a;  // 9
```

5. Wrapper Classes

Primitive Type	Wrapper Class
int	Integer
double	Double
char	Character
boolean	Boolean

6. Boxing & Unboxing

```
Integer n = 10;  // boxing (auto)
int value = n;   // unboxing (auto)
```

7. Important Note

Primitive variables store values; reference variables point to objects.

Java Important Cheat Sheet

Operators & Control Flow

1. Operators

Type	Operators
Arithmetic	+ - * / %
Comparison	== != > < >= <=
Logical	&& || !
Assignment	= += -= *= /=
Unary	++ -- + - !
Ternary	condition ? value1 : value2

2. if / else

```
int age = 20;
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

3. switch Expression

```
String type = switch (day) {
    case 1, 7 → "Weekend";
    default → "Weekday";
};
```

4. Loops

```
// for loop
for (int i = 0; i < 3; i++) {
    System.out.println(i);
}
// enhanced for loop (for-each)
for (int n : numbers) {
    System.out.println(n);
}
// while loop
while (condition) {
    // statements
}
// do-while loop
do {
    // statements
} while (condition);
```

5. Loop Control

Statement	Purpose
break	exit loop
continue	skip iteration
return	exit method

6. Important Notes

- Integer division: 5 / 2 → 2
- Use 5.0 / 2 → 2.5
- Short-circuit operators: && and ||.

Java Important Cheat Sheet

Strings & StringBuilder

1. Creating Strings

```
String s = "Java";
String t = new String("Java");
```

2. Access & Length

```
s.length()  → 4
s.charAt(0) → 'J'
s.substring(1, 3) → "av"
```

3. Search & Compare

```
s.contains("av") → true
s.indexOf("a") → 1
s.startsWith("Ja") → true
s.endsWith("va") → true
s.equals("Java") → true
s.equalsIgnoreCase("java") → true

Use equals(), not ==, for content.
```

4. Transform

```
s.toUpperCase() → "JAVA"
s.toLowerCase() → "java"
s.replace("J", "K") → "Kava"
" Java ".trim() → "Java"
" Java ".strip() → "Java"
```

5. Split & Join

```
"a,b,c".split(",") → ["a", "b", "c"]
String.join("-", "a","b","c") → "a-b-c"
```

6. Formatting

```
String.format("Age: %d", 25) → "Age: 25"
"Name: %s".formatted("Aman") → "Name: Aman"
```

7. StringBuilder

```
StringBuilder sb = new StringBuilder("Java");
sb.append(" 21");
sb.insert(0, "Learn ");
sb.replace(0, 5, "Use");
sb.delete(0, 4);
sb.reverse();
sb.toString();
```

8. Important Note

Strings are immutable; StringBuilder is mutable and efficient for repeated changes.

Java Important Cheat Sheet

Arrays & Arrays Utility

1. Create Arrays

```
int[] a = {4, 1, 7, 2};
int[] b = new int[5];
String[] names = {"Aman", "Riya"};
```

2. Access & Update

```
a[0] → 4
a[1] = 10;
a.length → 4
```

3. Traverse

```
for (int i = 0; i < a.length; i++) {
    System.out.println(a[i]);
}

for (int value : a) {
    System.out.println(value);
}
```

4. 2D Array

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
matrix[1][2] → 6
```

1	2	3
4	5	6

5. Arrays Utility Methods

```
Arrays.sort(a);
Arrays.binarySearch(a, 7);
Arrays.fill(b, 0);
Arrays.copyOf(a, 6);
Arrays.equals(a, b);
Arrays.toString(a);
Arrays.stream(a);
```

6. Copying

```
int[] copy = a.clone();
System.arraycopy(a, 0, b, 0, a.length);
```

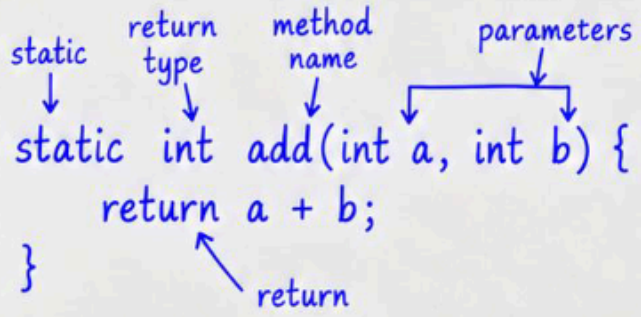
7. Important Notes

- Index starts at 0.
- Last index = length - 1.
- binarySearch() requires a sorted array.

Java Important Cheat Sheet

Methods & Parameters

1. Method Syntax



2. Calling a Method

```
int result = add(5, 3);  // 8
```

3. Parameters vs Arguments

Parameters — variables in definition: int a, int b

Arguments — actual values in call: 5, 3

4. Method Overloading

```
int sum(int a, int b) { ... }
double sum(double a, double b) { ... }
```

Same name, different parameter list.

5. Varargs

```
static int total(int... nums) {
    ...
}
total(10, 20, 30);
```

Only one varargs parameter; it must be last.

6. Pass-by-Value

Java always passes arguments by value.

- Primitive → copied value
- Object → copied reference value

7. Scope

local — inside method/block
instance — belongs to object
static — belongs to class

8. Recursion

```
static int fact(int n) {
   return n <= 1 ? 1 : n * fact(n - 1);
}
fact(5) → 120
```

9. Important Note

Use clear names, small methods and one responsibility.

Java Important Cheat Sheet

Classes, Objects & Constructors

1. Class & Object

```
class Car {
    String brand;
    int speed;
    void drive() {
        System.out.println(brand);
    }
}
Car car = new Car();
car.brand = "Tata";
car.drive();
```

Class → blueprint (defines fields and methods)

Object → instance (actual object with state)

2. Constructor

```
class User {
    String name;
    User(String name) {
        this.name = name;
    }
}
User u = new User("Aman");
```

- Same name as class.
- No return type.
- Runs during object creation.

3. Constructor Overloading

```
class User {
    User() {
        this("Guest");
    }
    User(String name) {
        this.name = name;
    }
}
```

- this(...) calls another constructor in the same class.
- It must be the first statement in the constructor.

4. this Keyword

this.name — current object's field
this(...) — another constructor
this.method() — current object's method

5. Static vs Instance

Feature	Instance (Non-static)	Static
Field	One copy per object	One copy per class
Method	Called using object	Called using class
Access	Through object	Through class name
Example	obj.field obj.method()	Class.field Class.method()

6. Encapsulation

```
class Person {
    private int age;
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        if (age >= 0) this.age = age;
    }
}
```

7. Imputable Class Checklist

- Class is final.
- Fields are private final.
- Initialize through constructor.
- No setters.
- Return defensive copies for mutable data.

8. Important Note

A class defines structure and behavior; objects hold actual state.

Java Important Cheat Sheet

Inheritance & Polymorphism

1. Inheritance

Child class (Dog) extends parent class (Animal) and inherits fields and methods.

```
class Animal {
    String name;

    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

2. super keyword

super() calls parent constructor.
super.method() calls parent class method.

```
class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    Dog() {
        super();      // calls parent constructor
    }
    void sound() {
        super.sound();  // calls parent method
        System.out.println("Dog sound");
    }
}
```

3. Method Overriding

Child class provides its own implementation with the same method signature.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog sound");
    }
}
```

4. Runtime Polymorphism

Parent reference refers to child object.
The overridden method is called at runtime.

```
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();  // upcasting
        a.sound();       // calls Dog's sound()
    }
}
```

5. Overloading vs Overriding

Method Overloading

- Same method name
- Different parameters
- Same class
- Compile-time (static)

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Method Overriding

- Same method name
- Same parameters
- Parent and child class
- Runtime (dynamic)

```
class Animal {
    void sound() {
        System.out.println("Animal");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog");
    }
}
```

6. final class / final method

- final class cannot be extended.
- final method cannot be overridden.

```
final class Animal {
    // cannot be extended
}
class Parent {
    final void show() {
        System.out.println("Final method");
    }
}
class Child extends Parent {
    // void show() { }   // Compile-time error
```

Bottom Takeaway

Inheritance reuses behavior; polymorphism selects the runtime implementation.

Java Important Cheat Sheet

Abstraction & Interfaces

1. Abstract class syntax

```
abstract class Shape {
   abstract double area();
   void info() {
      System.out.println("This is a shape");
   }
}
```

2. Concrete subclass implementing area()

```
class Circle extends Shape {
   double radius;
   Circle(double r) {
      this.radius = r;
   }
   @Override
   double area() {
      return Math.PI * radius * radius;
   }
}
```

3. Interface syntax

```
interface Printable {
   void print();
}
```

4. implements example

```
class Document implements Printable {
   public void print() {
      System.out.println("Printing document");
   }
}
```

5. Abstract Class vs Interface

Feature	Abstract Class	Interface
Constructors	Yes	No
State (fields)	Can have instance variables	Only public static final (constants)
Multiple implementation	A class can extend only one abstract class	A class can implement multiple interfaces
Methods	Can have abstract and concrete methods	Abstract methods (plus default and static methods)

6. default and static interface methods

```
interface Logger {
   default void log(String msg) {
      System.out.println("Log: " + msg);
   }
   static void info() {
      System.out.println("Logger interface");
   }
}

class App implements Logger {
   // inherits default method
}
```

7. Access modifiers (summary)

Modifier	Same Class	Same Package	Subclass (different pkg)	Other Package
private	Yes	No	No	No
default (no modifier)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

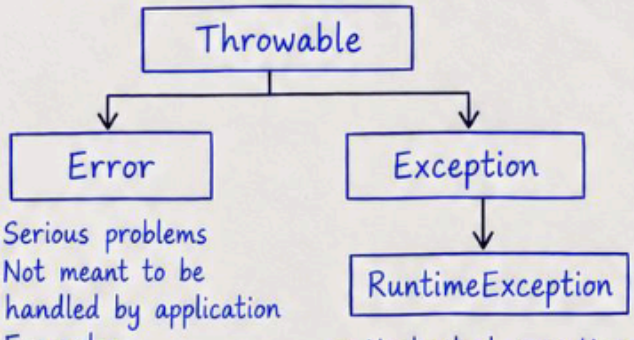
Bottom Takeaway:
Program to an interface, not an implementation.

Java Important Cheat Sheet

Exception Handling

@aman_views

1. Exception Hierarchy



- Serious problems
- Not meant to be handled by application
- Examples: OutOfMemoryError, StackOverflowError

- Unchecked exceptions
- Occur at runtime
- Examples: NullPointerException, ArithmeticException

2. try-catch-finally Example

```
try {
    int n = 10 / 0;
    System.out.println("Result: " + n);
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("This always runs");
}
```

3. Multiple catch and Catch Order

```
try {
    String s = null;
    int len = s.length();
} catch (NullPointerException e) {
    System.out.println("Null pointer");
} catch (Exception e) {   // more general
    System.out.println("Some other exception");
}
```

- Catch more specific exceptions first.
- General exceptions (like Exception) must come after specific ones.

4. throw vs throws

throw (used inside a method)

- Actually creates and throws an exception object.
- Used to explicitly throw an exception.

```
if (age < 18) {
    throw new IllegalArgumentException("Invalid age");
}
```

throws (used in method declaration)

- Declares that a method may throw an exception.
- Used for checked exceptions.

```
public void readFile() throws IOException {
    // may throw IOException
}
```

5. Checked vs Unchecked Exceptions

Checked Exceptions	Unchecked Exceptions
• Checked at compile time.	• Checked at runtime.
• Must be handled or declared with throws.	• Not required to be handled or declared.
• Examples: IOException, SQLException	• Examples: NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException

6. try-with-resources (Auto Close)

```
try (BufferedReader br = new BufferedReader(
                         new FileReader("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("Error reading file: " + e.getMessage());
}
```

7. Custom Exception

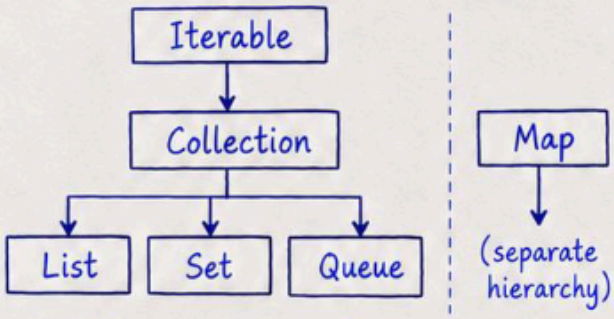
```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Bottom takeaway: Catch only what you can handle; never hide failures.

Java Important Cheat Sheet

Collections Framework

1. Hierarchy



- Iterable → root interface (for-each loop).
- Collection → parent of List, Set, Queue.
- Map is separate (not a subinterface of Collection).

2. List (Ordered, allows duplicates)

- Maintains insertion order.
- Allows duplicate elements.
- Common implementations:

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");

List<Integer> nums = new LinkedList<>();
nums.add(10);
nums.add(20);
```

3. Set (Unique elements)

- Does not allow duplicate elements.
- Types:

```
Set<String> set1 = new HashSet<>();
// no order, fast
Set<String> set2 = new LinkedHashSet<>();
// maintains insertion order
Set<String> set3 = new TreeSet<>();
// sorted order (natural ordering)
```

4. Queue / Deque (Processing order)

- Queue → typically FIFO (First In First Out).
- Deque → double-ended queue (add/remove from both ends).
- Types:

```
Queue<Integer> q = new PriorityQueue<>();
// elements processed by natural ordering
q.add(5);
q.add(1);
q.add(3);   // poll() returns 1

Deque<String> dq = new ArrayDeque<>();
dq.addFirst("A");
dq.addLast("B");
String first = dq.removeFirst();
```

5. Map (Key-Value, unique keys)

- Stores key-value pairs.
- Keys are unique (values can be duplicate).
- Types:

```
Map<String, Integer> map1 = new HashMap<>();
// no order, fast
Map<String, Integer> map2 = new LinkedHashMap<>();
// maintains insertion order
Map<String, Integer> map3 = new TreeMap<>();
// sorted by keys (natural ordering)
```

6. Quick Selection Guide

- Need ordered elements (duplicates allowed)? → List
- Need unique elements? → Set
- Need FIFO processing? → Queue
- Need priority-based processing? → PriorityQueue
- Need add/remove from both ends? → Deque
- Need key-value mapping? → Map
- Need sorted elements/keys? → TreeSet or TreeMap
- Need insertion order? → LinkedHashSet or LinkedHashMap

7. Note

- Collections store objects (reference types).
- For primitive data, use wrapper classes: int → Integer, long → Long, double → Double, etc.

Bottom Takeaway: Choose by order, uniqueness, lookup, and update cost.

Java Important Cheat Sheet

@aman_views

List, Set, Map & Queue Methods

1. List (ArrayList)

```
List<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
list.add(30);          // [10, 20, 30]
list.get(1)            // 20
list.set(1, 25);       // [10, 25, 30]
list.remove(0);        // [25, 30]
list.contains(30);     // true
list.size();           // 2
```

2. Set (HashSet)

```
Set<String> set = new HashSet<>();
set.add("apple");      // true
set.add("banana");     // true
set.add("apple");      // false (duplicate)
set.contains("banana");// true
set.remove("apple");   // true
set.size();            // 1
```

Set ignores duplicate elements.

3. Map (HashMap)

```
Map<String, Integer> map = new HashMap<>();
map.put("a", 1);
map.put("b", 2);
map.put("c", 3);
map.get("b");              // 2
map.getOrDefault("d", 0); // 0
map.containsKey("a");      // true
map.remove("b");           // 2 (removed value)
map.keySet();              // [a, c]
map.values();              // [1, 3]
map.entrySet();            // [a=1, c=3]
```

4. Queue (ArrayDeque)

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(10);    // true (add to end)
q.offer(20);    // true
q.offer(30);    // true
q.peek();       // 10  (front element)
q.poll();       // 10  (removes and returns)
q.peek();       // 20
q.size();       // 2
```

5. Useful Collections Methods

```
List<Integer> list = new ArrayList<>();
list.add(3);
list.add(1);
list.add(2);
Collections.sort(list);    // [1, 2, 3]
Collections.reverse(list); // [3, 2, 1]
Collections.max(list);     // 3
Collections.min(list);     // 1
```

6. Arrays.asList()

```
List<String> names = Arrays.asList("A", "B", "C");
names.get(0);      // "A"
names.size();      // 3
```

Returns a fixed-size list backed by the array.
Use new ArrayList<>(...) if you need to modify.

7. Important Note

```
List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);
list.remove(1);     // removes element at index 1
                    // list is now [1]
list.remove(Integer.valueOf(1)); // removes value 1
// removes the first occurrence of 1
```

8. Bottom Line

Use interface types on the left side of declarations.

```
List<Integer> list = new ArrayList<>();
Set<String> set = new HashSet<>();
Map<String, Integer> map = new HashMap<>();
Queue<Integer> q = new ArrayDeque<>();
```

Java Important Cheat Sheet

Generics, Lambdas & Streams

1. Generic Class

```
class Box<T> {
    T value;

    void set(T v) {
        this.value = v;
    }

    T get() {
        return value;
    }
}
```

2. Generic Method

```
static <T> void print(T value) {
    System.out.println(value);
}
// Can be called with any type
print(10);
print("Hello");
```

3. Bounds and Wildcards

```
// Bounded type parameter
class NumberBox<T extends Number> {
    T value;
    NumberBox(T value) {
        this.value = value;
    }
}

// Wildcards
List<?> anyList;          // unknown type
List<? extends Number> nums; // T or subtype
List<? super Integer> ints;  // T or supertype
```

4. Functional Interface

```
@FunctionalInterface
interface MyFunction<T, R> {
    R apply(T t);
}
// Represents a function that takes T and
// returns R. Can be implemented using a
// lambda expression.
```

5. Lambda Examples

```
// Add two numbers
BiFunction<Integer, Integer, Integer> add =
    (a, b) -> a + b;
int sum = add.apply(3, 5);   // 8

// Print a name
Consumer<String> printer =
    name -> System.out.println(name);
printer.accept("Aman");
```

6. Stream Pipeline

```
// Filter, map, sort and collect to a list
List<Integer> nums = List.of(5, 2, 9, 1, 6);
List<Integer> result = nums.stream()
        .filter(n -> n % 2 == 0) // even numbers
        .map(n -> n * 2)         // multiply by 2
        .sorted()                // ascending
        .toList();
```

7. Common Terminal Operations

```
forEach(x -> System.out.println(x)); // no return
collect(Collectors.toList());        // to a list
reduce(0, Integer::sum);             // combine
count();                             // total count
anyMatch(n -> n > 0);                // true if any
```

8. Streams Do Not Modify Source

```
List<Integer> nums = List.of(1, 2, 3);
nums.stream().map(n -> n * 2).toList();
// nums remains unchanged: [1, 2, 3]
```

Bottom Takeaway: Generics add type safety; streams express data pipelines.

Java Important Cheat Sheet

@aman_views

File I/O, Multithreading & JVM

1. File I/O (Reading & Writing Files)

a) Reading a file

```
Path path = Path.of("data.txt");
String content = Files.readString(path);
```

b) Writing to a file

```
Path path = Path.of("data.txt");
Files.writeString(path, "Hello Java!");
```

c) Try-with-resources (auto close)

```
try (BufferedReader br =
    Files.newBufferedReader(Path.of("data.txt"))) {
  String line;
  while ((line = br.readLine()) != null) {
    System.out.println(line);
  }
} catch (IOException e) {
  e.printStackTrace();
}
```

2. Multithreading

a) Creating and starting a thread

```
Thread t = new Thread(() ->
      System.out.println("Run"));
t.start();
```

b) start() vs run()

- start() creates a new thread and calls run().
- run() runs on the current thread (no new thread).

c) Synchronized (thread safety)

```
class Counter {
  private int count = 0;
  public synchronized void increment() {
    count++;
  }
```

2. Multithreading (contd.)

d) Using ExecutorService

```
ExecutorService pool =
    Executors.newFixedThreadPool(2);
pool.submit(() -> System.out.println("Task 1"));
pool.submit(() -> System.out.println("Task 2"));
pool.shutdown();
```

3. JVM Essentials

a) JDK, JRE and JVM

- JDK = tools + JRE (compiler, debugger, etc.)
- JRE = JVM + standard libraries
- JVM runs bytecode (.class files) on the platform.

b) Memory Areas: Stack vs Heap

- Stack: method calls, local variables (per thread). Fast and automatically managed.
- Heap: objects and arrays. Shared by all threads. Managed by garbage collector.

c) Garbage Collection

- Automatically reclaims memory of unreachable objects.
- Helps prevent memory leaks.

d) String Pool

- String literals are stored in a special pool in heap.
- Identical literals refer to the same object.

```
String a = "Hello";
String b = "Hello";
System.out.println(a == b);  // true (from pool)
```

Best Practices

1. Close resources using try-with-resources.
2. Prefer executors over creating raw threads.
3. Avoid shared mutable state between threads.
4. Use immutable objects where possible.

Bottom line: Write clear code, handle resources safely, and understand runtime behavior.